

CSC 307 & CSC 308 — Operating Systems

Bells University of Technology, Ota

Department of Computer Science & Information Technology

Comprehensive Study Notes, Summaries & Model Answers

SECTION 1: CSC 307 — Operating Systems II

This section covers exam papers from 2022/2023 and 2023/2024, with full model answers for every question.

1.1 — Key Topic Summary

Design Requirements of an Operating System

There are two parts to OS design requirements:

- User Goals: convenience, ease of learning, reliability, safety, and speed.
- System Goals: ease of design, implementation, and maintenance; flexibility; efficiency; and error-freedom.

Five Design Goals of a Modern OS

- Reliability — the system should function correctly and consistently without crashing.
- Performance — efficient use of CPU, memory, and I/O resources.
- Portability — able to run across different hardware platforms.
- Security — protection against unauthorized access and malicious attacks.
- Scalability — ability to handle growing workloads and users.

Five Important Issues in Analyzing & Designing an OS

- Efficiency vs. Fairness: Maximizing CPU use for one process can starve others. Solved by scheduling algorithms (e.g., Round Robin, CFS).
- Modularity vs. Performance: Separating OS into modules improves maintainability but can add overhead (e.g., microkernels vs. monolithic).
- Security vs. Convenience: Strong security measures (authentication, encryption) can slow the user experience.
- Generality vs. Specialization: A general-purpose OS serves many users, but specialized systems (real-time, embedded) need tailored design.
- Naming and Protection: How resources (files, processes, memory) are named and who is allowed to access them must be carefully managed.

Input/Output Devices — Two Categories

- Block Devices: Transfer data in fixed-size blocks; support random access (e.g., hard disks, SSDs, USB drives).
- Character Devices: Transfer data one character/byte at a time in a stream; no random access (e.g., keyboards, mice, serial ports, printers).

Three Approaches the OS Uses to Communicate with I/O Devices

- Programmed I/O (Polling): The CPU continuously checks ('polls') whether the device is ready. Simple but wastes CPU time.
- Interrupt-Driven I/O: The device sends an interrupt signal when ready; the CPU handles other work in between. More efficient.
- Direct Memory Access (DMA): A DMA controller transfers data directly between device and memory, freeing the CPU entirely during transfers.

Three Main Levels of I/O Software

- Interrupt Handlers: Low-level routines that respond immediately when a device signals completion.
- Device Drivers: Software specific to each device type that translates OS commands into device-specific operations.
- Device-Independent I/O Software: Provides a uniform interface to upper layers, handles buffering, error reporting, and naming.

Physical vs. Logical Address Space

Physical Address: The actual address in RAM hardware. Set by the Memory Management Unit (MMU).

Logical (Virtual) Address: The address generated by the CPU for a process. The process sees only logical addresses; the OS/MMU translates them to physical ones. This enables memory protection and virtual memory.

Memory Allocation — Two Major Types

- **Contiguous Allocation:** Each process occupies a single, continuous block of memory. Simple but causes fragmentation.
- **Non-Contiguous Allocation:** A process's memory is split across different locations (using paging or segmentation). More flexible; avoids external fragmentation.

Fixed vs. Variable Partition (3 each)

Fixed Partition:

- Memory is divided into fixed-size partitions at system startup.
- Each partition holds exactly one process, regardless of size.
- Causes internal fragmentation (wasted space inside a partition).

Variable Partition:

- Partitions are created dynamically to match the exact size of each process.
- Reduces internal fragmentation but causes external fragmentation over time.
- Requires compaction periodically to consolidate free memory.

Four Advantages of Variable Partitioning over Fixed Partitioning

- No internal fragmentation — partitions match process size exactly.
- Better memory utilization — more processes can be loaded simultaneously.
- Flexibility — supports processes of varying sizes without pre-planning.
- Easier growth — a process can be given more memory without fixed limits.

Five Reasons for Using Memory Management

- **Relocation:** Programs can be loaded at any address; the OS translates addresses.
- **Protection:** Prevents one process from accessing another's memory space.
- **Sharing:** Allows multiple processes to share code (e.g., shared libraries).
- **Logical Organization:** Supports the programmer's view of memory as a linear space.
- **Physical Organization:** Manages the two-level hierarchy of RAM and secondary storage.

Virtual Memory — Demand Paging and Segmentation

Demand Paging:

Pages of a process are loaded into memory only when they are needed (on demand), not all at once. If a required page is not in RAM, a page fault occurs and the OS fetches it from disk. This allows programs larger than physical RAM to run.

Segmentation:

Memory is divided into variable-length segments corresponding to logical divisions of a program (e.g., code segment, data segment, stack segment). Each segment has a name and a length, and is mapped independently to physical memory.

Advantages and Disadvantages of Virtual Memory

Advantages:

- Programs larger than physical RAM can execute.
- More processes can be kept in memory simultaneously, improving CPU utilization.

Disadvantages:

- Page faults cause disk I/O, which is slow — too many faults cause 'thrashing'.

- Increases OS complexity and overhead for managing page tables.

Four Popular Scheduling Algorithms

- First-Come, First-Served (FCFS): Jobs are processed in arrival order. Simple but can cause convoy effect.
- Shortest Job First (SJF): Runs the process with the shortest burst time next. Minimizes average waiting time but requires knowing burst times in advance.
- Round Robin (RR): Each process gets a fixed time slice (quantum); preemptive. Fair, good for interactive systems.
- Priority Scheduling: Processes are given priority levels; the highest priority runs first. Can cause starvation of low-priority processes (solved by aging).

Four Measures to Minimize Cyber Threat Risks

- Use strong, unique passwords and enable multi-factor authentication.
- Keep operating systems and software updated with security patches.
- Use firewalls and antivirus/antimalware software.
- Regularly back up data and limit user privileges (principle of least privilege).

Three Principles of Protection

- Principle of Least Privilege: Give each process/user only the minimum access rights needed to perform their task.
- Separation of Policy and Mechanism: Keep what is protected (policy) separate from how protection is enforced (mechanism), enabling flexibility.
- Principle of Complete Mediation: Every access to every resource must be checked against the protection policy — no exceptions.

Three Resource Allocation Techniques

- Resource Allocation Graph (RAG): A directed graph showing which processes hold which resources and which are waiting. Cycles indicate deadlock.
- Banker's Algorithm: Processes declare maximum resource needs in advance; the OS only grants requests that keep the system in a 'safe state'.
- Preemption: Resources are forcibly taken from a process and reallocated to another when needed, breaking potential deadlocks.

One Advantage of Each Resource Allocation Technique

- RAG: Simple visual detection of deadlock cycles without complex computation.
- Banker's Algorithm: Proactively prevents deadlock by ensuring a safe state is always maintained.
- Preemption: Can resolve deadlocks dynamically without process termination.

Three Types of Schedulers

- Long-Term Scheduler (Job Scheduler): Decides which processes are admitted into the ready queue from the job pool. Controls degree of multiprogramming. Runs infrequently.
- Short-Term Scheduler (CPU Scheduler): Selects which ready process gets the CPU next. Runs very frequently (milliseconds). Must be fast.
- Medium-Term Scheduler (Swapper): Temporarily removes processes from memory (swapping) to reduce degree of multiprogramming and improve performance.

Distributed Operating System — Four Design Issues Handled

- Transparency: The distributed nature of the system is hidden from users; they see a single unified system.
- Fault Tolerance: The system continues operating even when individual nodes fail, through redundancy and replication.
- Scalability: The system can grow by adding more nodes without restructuring the whole system.

- Security: Distributed systems must authenticate users and protect data across network boundaries, using encryption and access control.
-

SECTION 2: CSC 308 — Operating Systems (Group 4 & Q&A)

This section covers OS structures, system calls, kernel designs, and file systems from the CSC 308 material and Group 4 questions.

2.1 — OS Structure Questions & Answers

Q1. Distinguish between Simple Structure and Monolithic Structure.

Simple Structure (e.g., MS-DOS): No formal interfaces between subsystems — any component can call any other, and application programs can even access hardware directly. No separation of concerns. Monolithic Structure (e.g., original UNIX): All OS services run in a single kernel binary in privileged mode, but the system defines internal interfaces between subsystems (scheduler, memory manager, file system). Significance: In Simple Structure, a bug anywhere causes unpredictable effects. In Monolithic Structure, defined internal interfaces constrain faults, making them more traceable — though a bug in any component can still crash the entire kernel.

Q2. What are the two categories of OS design requirements?

1. User Goals: Users want the OS to be convenient, easy to learn, reliable, safe, and fast. Examples: quick response time (performance), no crashes during normal use (reliability). 2. System Goals: Designers want the OS to be easy to design, implement, and maintain; flexible; efficient; and error-free. Examples: portability across hardware, extensibility without restructuring. These categories frequently conflict — more features improve convenience but increase complexity. This drives the fundamental principle of separating mechanism (how) from policy (what).

Q3. Explain how a system call transfers control from user mode to kernel mode.

1. A user program calls a library wrapper (e.g., `read()`). The wrapper places the system call number in a CPU register (RAX on x86-64) and arguments in other registers. 2. The wrapper executes a privileged trap instruction (`syscall` on x86-64, `SVC` on ARM), triggering a CPU mode switch from user mode (Ring 3) to kernel mode (Ring 0). 3. The CPU saves the program counter, stack pointer, and CPU flags onto the kernel stack, then transfers control to the OS system call dispatcher. 4. The dispatcher indexes the system call table using the call number and invokes the correct kernel function after validating arguments. 5. The kernel executes the service with full hardware access. 6. On completion, the kernel places the return value in RAX and executes `sysret`, switching back to user mode.

Q4. Compare Microkernel and Modular structures.

Microkernel: Keeps only IPC, elementary memory management, and low-level scheduling in kernel space. All other services run as user-space server processes communicating via message passing (e.g., Mach, QNX). Modular: Maintains a small kernel core with functionality added through dynamically loadable kernel modules (LKMs) running in kernel mode with direct function calls (e.g., Linux, Solaris). Key differences — Performance: Modular is faster (direct calls); Microkernel has IPC overhead. Fault isolation: Microkernel is stronger (crashed user-space server doesn't crash kernel). Choose Microkernel for safety-critical systems (medical, avionics). Choose Modular for general-purpose computing (Linux, servers, Android).

Q5. Identify the three components of the XNU kernel (macOS/iOS).

1. Mach Microkernel: Provides fundamental abstractions — tasks, threads, ports, and messages. Also manages virtual memory with copy-on-write sharing. Structural model: Microkernel. 2. BSD Subsystem: Derived from FreeBSD. Provides POSIX-compatible system calls, TCP/IP networking stack, and Virtual File System (VFS). Runs in kernel space for performance. Structural model: Monolithic. 3. I/O Kit Device Driver Framework: A C++ framework supporting drivers as either user-space processes (fault isolation) or kernel extensions/kexts (performance). Structural model: Modular. Together they make XNU genuinely hybrid.

Q6. What are the two levels at which OS services exist? List three services each.

Level 1 — User-Oriented Services: (i) User Interface (CLI or GUI), (ii) Program Execution (loading, running, terminating programs), (iii) File-System Manipulation (creating, reading, writing, deleting files). Level 2 — System-Oriented Services: (i) Resource Allocation (distributing CPU, memory, and I/O among concurrent processes), (ii) Accounting and Logging (recording resource use for billing and auditing), (iii) Protection and Security (controlling access and defending against threats).

Q7. Explain the Android OS architecture. Why is it classified as a hybrid OS?

Android has 5 layers: (1) Linux Kernel — process management, device drivers, security using modular monolithic model. (2) Native C/C++ Libraries — performance-critical services (libc, OpenSSL, SQLite). (3) Android Runtime (ART) — executes Java/Kotlin apps using Ahead-Of-Time (AOT) compilation. (4) Java API Framework — Activity Manager, Window Manager, Content Providers. (5) Applications — user-facing apps. It is hybrid because it combines the modular monolithic model (Linux kernel), layered model (framework layers), and virtual machine abstraction (ART) — no single structural paradigm describes the full stack.

Q8. Trace the complete OS boot sequence from power-on to user applications running.

Stage 1 — UEFI Firmware: CPU executes ROM program (UEFI), performs Power-On Self-Test (POST), initialises hardware, locates bootable device. Stage 2 — Bootloader: UEFI loads bootloader (GRUB on Linux, Windows Boot Manager on Windows), which loads the OS kernel into RAM. Stage 3 — Kernel Initialisation: Kernel initialises internal data structures, detects/initialises hardware, mounts root file system. Stage 4 — First User-Space Process: Kernel launches init or systemd (Linux) / wininit.exe (Windows), which starts all system daemons and services. Only then can user applications execute.

Q9. What is System Generation (SYSGEN)? How do modern OSes handle this problem?

SYSGEN is the process of configuring an OS for a specific hardware installation. Historically (mainframes), administrators manually selected OS components and compiled the OS from source for their hardware. Modern approach: The modular structure solves this. The kernel contains a small core; optional components (device drivers, file systems, network protocols) are dynamically loadable modules (LKMs on Linux). At boot, the OS auto-detects hardware and loads only the required modules — no manual compilation needed.

Q10. Why have hybrid OS structures become dominant in modern computing?

Every pure structural model has an irreducible limitation: Monolithic is fast but a single bug crashes the whole system; Layered is clean but has performance overhead; Microkernel has strong fault isolation but IPC overhead is prohibitive for general-purpose use; Modular is flexible but lacks user-space fault isolation. Hybrid structures apply each model where it excels. Examples: macOS/iOS XNU (Mach + BSD + I/O Kit), Linux (monolithic core + loadable modules), Windows (microkernel + modular executive + HAL). Pure microkernels (QNX, MINIX 3) remain right for safety-critical embedded systems where isolation matters more than performance.

2.2 — CSC 308 File Systems & Security Q&A

Q1. What is a file system and why is it important?

A file system is a method used by an OS to organize, store, and retrieve data on storage devices. It is important because it prevents data corruption, improves storage efficiency, and provides an organized structure for files and directories.

Q2. Contiguous vs. Linked Allocation in file systems.

Contiguous allocation stores files in consecutive disk blocks — provides fast sequential and random access but causes external fragmentation. Linked allocation stores file blocks in different locations connected by pointers — reduces fragmentation but access to arbitrary blocks is slow (must follow pointer chain).

Q3. Protection vs. Security in OS.

Protection controls how processes and users access system resources (internal mechanisms like permissions and ACLs). Security protects the entire system from external threats such as hackers, malware, and unauthorized network access.

Q4. The CIA Triad.

Confidentiality: Prevents unauthorized parties from accessing data. Integrity: Ensures data remains accurate and has not been tampered with. Availability: Ensures systems and data are accessible to authorized users when needed.

Q5. Role of device drivers in an I/O system.

Device drivers act as translators between the operating system and hardware devices. They convert generic OS I/O commands into device-specific instructions, enabling the OS to communicate uniformly with printers, keyboards, disks, and other peripherals.

Q6. Polling vs. Interrupt-Driven I/O.

Polling: The CPU continuously checks the device's status register to see if it is ready — wastes CPU cycles while waiting. Interrupt-Driven I/O: The device sends a hardware interrupt signal when it is ready; the CPU handles other tasks in between and only responds when interrupted — far more efficient.

Q7. What is DMA and why is it useful?

Direct Memory Access (DMA) allows I/O devices to transfer data directly to or from main memory without continuous CPU involvement. A DMA controller handles the transfer; the CPU only needs to initiate it and is notified on completion. This dramatically improves throughput for large data transfers (e.g., disk reads) and reduces CPU workload.

Q8. Define a distributed system.

A distributed system is a collection of independent computers connected through a network that work together and appear to users as a single, unified, coherent system. Examples include cloud computing platforms and the internet.

Q9. Two advantages and two disadvantages of distributed systems.

Advantages: (1) High availability — failure of one node does not bring down the whole system. (2) Scalability — capacity can be increased by adding more nodes. Disadvantages: (1) Increased complexity — designing, debugging, and maintaining distributed systems is difficult. (2) Network dependency — the system relies on network connectivity; network failures can render services inaccessible.

Q10. Client-Server vs. Peer-to-Peer architecture.

Client-Server: Centralized model where dedicated server machines provide services and clients request them. Servers are always available. Example: web servers (HTTP). Peer-to-Peer (P2P): Decentralized model where each node (peer) can act as both client and server simultaneously. No central authority. Example: BitTorrent, blockchain networks.

SECTION 3: Group 1 — CSC 308 OS Concepts Q&A

These questions were assigned to Group 2 and cover core OS concepts, process management, scheduling, deadlock, and memory.

Q1. Define an OS and explain its dual role as resource manager and control program.

An Operating System is system software that manages computer hardware and software resources and provides an environment for programs to run. As a resource manager, it allocates the CPU, memory, storage, and I/O devices efficiently among competing processes. As a control program, it supervises program execution to prevent errors and misuse of the computer, ensuring processes do not interfere with each other or with the OS itself.

Q2. How do Process Management, Memory Management, and I/O Management interact?

Process management handles the creation, scheduling, and execution of programs. Memory management allocates RAM to processes so they have space to execute. I/O management provides processes with access to peripheral devices. Together: when a process is created (process management), it is assigned memory (memory management), and during execution it communicates with devices (I/O management). They are interdependent — a process cannot run without memory, and I/O operations cause process state transitions (running to waiting).

Q3. When do efficiency and fairness conflict?

Giving one process more CPU time improves efficiency (it finishes faster) but reduces fairness (other processes wait longer or may starve). For example, always scheduling the shortest job first (SJF) is efficient but starves long jobs. The OS resolves this using algorithms like Round Robin (equal time slices) or priority scheduling with aging (low-priority processes gradually gain priority over time) to balance both concerns.

Q4. Compare Batch OS and Time-Sharing OS with modern relevance.

Batch OS collects jobs into batches and processes them sequentially without user interaction — efficient for repetitive large-scale tasks. Time-Sharing OS allows multiple users to interact with the system simultaneously by rapidly switching the CPU between users. Batch systems remain relevant today in payroll processing, scientific simulations, and nightly report generation. Time-sharing is the foundation of all modern interactive systems (desktops, web servers).

Q5. How do kernel structures affect performance and security?

Monolithic kernels run all OS services in a single address space — fast due to direct function calls with no message-passing overhead, but less secure because a bug in any component (even a device driver) can crash the entire kernel. Microkernels isolate services as user-space processes — more secure and fault-tolerant (a crashed service doesn't bring down the kernel), but slower due to IPC overhead for inter-service communication. Hybrid kernels aim to balance both, though they inherit some complexity from each approach.

Q6. Evaluate hybrid kernels.

Hybrid kernels (e.g., Windows NT, macOS XNU) combine the performance benefits of monolithic design with the modularity and isolation benefits of microkernels. For example, performance-critical services run in kernel space while other services are modular and loadable. However, they may still inherit the complexity and partial security weaknesses of both pure designs. In practice, they have proven to be the dominant real-world OS architecture due to their practical balance.

Q7. Explain abstraction in OS.

Abstraction hides the complexity of hardware from application programs and users. The file system abstracts away the physical structure of disks — a program sees files and folders, not disk sectors and block addresses. Virtual memory abstracts physical RAM — a program sees a large, flat, private address space regardless of how much RAM is actually installed or how it is arranged. Abstraction simplifies programming and improves portability.

Q8. Explain PCB in context switching.

The Process Control Block (PCB) is a data structure the OS maintains for each process, storing its state: program counter, CPU registers, memory maps, open file descriptors, scheduling info, and I/O status. During a context switch, the OS saves the running process's state into its PCB (so it can be resumed later) and loads the state of the next process from its PCB into the CPU. This allows multiple processes to share a single CPU by taking turns.

Q9. Explain the Running to Waiting transition.

This process state transition occurs when a running process requests an I/O operation (e.g., reads from disk, waits for user input). Since I/O is much slower than the CPU, the OS moves the process from the Running state to the Waiting (Blocked) state and schedules another process. When the I/O completes, the OS moves the process back to the Ready queue so it can be scheduled again.

Q10. How does the OS ensure CPU fairness?

The OS uses scheduling algorithms: Round Robin assigns each process an equal time quantum and cycles through the ready queue — no process is ignored. Priority Scheduling with Aging gradually increases the priority of waiting processes so low-priority jobs eventually run. The Completely Fair Scheduler (CFS) used in Linux distributes CPU time proportionally based on each process's weight, ensuring every process receives its fair share over time.

Q11. What is a system call and how does it differ from a library function?

A system call is a programmatic request from a user process to the OS kernel for a privileged service — such as creating a process (`fork()`), reading a file (`read()`), or allocating memory (`brk()`). It triggers a mode switch from user mode to kernel mode. A library function (e.g., `printf()`, `strlen()`) is a user-space routine that runs in the same mode as the calling program. Library functions may or may not invoke system calls internally (e.g., `fread()` eventually calls the `read()` system call, but `strlen()` does not).

Q12. Preemptive vs. Non-Preemptive Scheduling.

Preemptive Scheduling: The OS can forcibly remove a process from the CPU when its time quantum expires or a higher-priority process arrives. Used in most modern OSes — ensures responsiveness and fairness. Examples: Round Robin, Priority with preemption. **Non-Preemptive Scheduling:** A process keeps the CPU until it voluntarily releases it (via I/O request, yield, or termination). Simpler but can cause long waiting times. Example: FCFS, non-preemptive SJF.

Q13. What is a deadlock? Name three necessary conditions.

A deadlock is a situation where two or more processes are permanently blocked, each waiting for a resource held by another process in the set — creating a circular dependency with no process able to proceed. Three necessary conditions (Coffman conditions): (1) Mutual Exclusion — at least one resource cannot be shared; only one process uses it at a time. (2) Hold and Wait — a process holds at least one resource while waiting to acquire additional resources. (3) Circular Wait — there exists a closed chain of processes, each waiting for a resource held by the next.

Q14. Compare thread vs. process. When is multithreading preferred?

A process is an independent program in execution with its own memory space, file descriptors, and system resources — heavyweight, isolated. A thread is a lightweight unit of execution within a process; threads share the memory space and resources of their parent process. Multithreading is preferred when: tasks share data and need to communicate frequently (shared memory is faster than IPC); the overhead of creating separate processes is undesirable; tasks are I/O-bound and can make progress while others wait (e.g., a web server handling multiple client requests). Multiprocessing is preferred when tasks need true isolation (e.g., separate security domains) or when parallelism across multiple CPU cores without sharing is needed.

SECTION 4: Group 6 — Memory Management & Virtual Memory (Practical)

This section contains worked solutions for all 10 memory management practical questions.

Paging & Address Translation

Q1. Paging: Page size = 4KB, Logical address = 13,000 bytes. Find page number, offset, frame, and physical address.

Page size = 4KB = 4096 bytes. (a) Page number = $\text{floor}(13000 / 4096) = 3$. Offset = $13000 - (3 \times 4096) = 13000 - 12288 = 712$ bytes. (b) From page table: Page 3 maps to Frame 1. (c) Physical address = $(\text{Frame} \times \text{Page size}) + \text{Offset} = (1 \times 4096) + 712 = 4808$ bytes.

Page Replacement Algorithms

Q2. FIFO with 3 frames, reference string: 7,0,1,2,0,3,0,4,2,3,0,3. Find total page faults and final frame state.

Simulation: 7→Fault[7] | 0→Fault[7,0] | 1→Fault[7,0,1] | 2→Replace 7→[2,0,1] | 0→Hit | 3→Replace 0→[2,3,1] | 0→Replace 1→[2,3,0] | 4→Replace 2→[4,3,0] | 2→Replace 3→[4,2,0] | 3→Replace 0→[4,2,3] | 0→Replace 4→[0,2,3] | 3→Hit Total Page Faults = 9 Final Frames = [0, 2, 3]

Q3. LRU with 3 frames, reference string: 1,2,3,4,1,2,5,1,2,3,4,5. Find page faults and final frame state.

LRU replaces the page that was least recently used. After full simulation: Total Page Faults = 10 Final Frames = [3, 4, 5]

Q4. Optimal Page Replacement with 3 frames, reference string: 2,3,2,1,5,2,4,5,3,2,5,2. Why does Optimal perform better?

Optimal replaces the page that will not be used for the longest time in the future. Total Page Faults = 7 Optimal performs better than FIFO/LRU because it makes the theoretically perfect replacement decision using future knowledge — minimizing future faults. In practice, Optimal cannot be implemented (future is unknown), but it serves as a benchmark to compare other algorithms against.

Memory Fit Algorithms

Q5. First Fit: Memory blocks = 100MB, 500MB, 200MB, 300MB, 600MB. Processes: P1=212MB, P2=417MB, P3=112MB, P4=426MB.

First Fit scans from the start and allocates the first block large enough: P1 (212MB) → 500MB block (internal fragmentation = 288MB) P2 (417MB) → 600MB block (internal fragmentation = 183MB) P3 (112MB) → 200MB block (internal fragmentation = 88MB) P4 (426MB) → No remaining block is large enough → NOT ALLOCATED Total internal fragmentation = $288 + 183 + 88 = 559$ MB

Q6. Compare First Fit vs. Best Fit using the same blocks and processes.

First Fit: Faster (stops at first suitable block). Higher fragmentation (559MB). Allocates 3 processes. Best Fit: Slower (scans all blocks to find smallest adequate one). Lower fragmentation (433MB). Also allocates 3 processes. Conclusion: Best Fit is more memory-efficient; First Fit is faster. Both fail to allocate P4 due to its size. Best Fit is preferred when memory conservation matters.

Effective Access Time

Q7. Memory access = 120ns, page fault rate = 2%, page fault service = 8,000ns. Compute EAT.

$EAT = (1 - p) \times \text{memory_access_time} + p \times \text{page_fault_service_time}$
 $EAT = (0.98 \times 120) + (0.02 \times 8000)$
 $EAT = 117.6 + 160$
 $EAT = 277.6 \text{ ns}$
Interpretation: Even a small 2% page fault rate more than doubles the effective access time (from 120ns to 277.6ns). This illustrates why minimizing page faults is critical for system performance.

Demand Paging

Q8. Demand paging: 10-page program, only 3 pages initially loaded. What happens when a required page is not in memory?

1. The CPU generates a logical address for the missing page. 2. The MMU detects the page is not in RAM — triggers a page fault (hardware interrupt). 3. The OS traps the fault, saves the process state. 4. The OS identifies which page is needed and locates it on disk. 5. If there is a free frame, the page is loaded into it. If not, a page replacement algorithm (FIFO, LRU, Optimal) selects a victim page to evict. 6. The page table is updated to reflect the new mapping. 7. The faulting instruction is restarted; the process resumes. Performance impact: Faster startup (fewer pages loaded initially), but frequent faults cause high disk I/O and can degrade performance significantly. Reducing the fault rate through good locality of reference is key.

Multiprogramming and RAM

Q9. 16GB RAM, 1GB reserved for OS, processes need 2GB each. Calculate processes supported; impact if process size increases to 4GB.

Available RAM = 16 - 1 = 15GB. Case 1 (2GB per process): $15 \div 2 = 7$ processes (integer division → 7 processes, 1GB unused). Case 2 (4GB per process): $15 \div 4 = 3$ processes (integer division → 3 processes, 3GB unused). Conclusion: Increasing process size from 2GB to 4GB reduces the multiprogramming level from 7 to 3, cutting CPU utilization significantly. This illustrates why memory efficiency matters for system throughput.

Q10. 20MB RAM, 50MB application. How does the OS execute it? How is data moved between RAM and disk?

The OS uses virtual memory with demand paging: 1. The OS does not load the entire 50MB program into RAM at once. Instead, the program is divided into 4KB pages. 2. Only the pages currently needed for execution are loaded into the 20MB RAM (which holds at most ~5,120 pages if all 20MB is available). 3. The rest of the program remains on secondary storage (the swap space on disk). 4. As execution proceeds, if the CPU accesses a page not in RAM, a page fault fires and the OS fetches that page from disk, replacing a less-needed page if RAM is full. Data movement: Frequently used pages stay in RAM. Pages that haven't been accessed recently are written back to disk (if modified) or simply evicted (if clean). This continuous, automatic swapping makes a 50MB program run on 20MB RAM — transparently to the application.

SECTION 5: Quick Reference — Key Definitions

- Operating System: System software managing hardware and software resources, providing an interface between user applications and hardware.
- Process: An independent program in execution with its own memory space, PCB, and system resources.
- Thread: A lightweight unit of execution within a process; shares the process's memory space.
- PCB (Process Control Block): OS data structure holding all information about a process — PC, registers, memory maps, open files.
- Context Switch: Saving the current process state (to PCB) and restoring another process's state (from PCB) to switch CPU between processes.
- Virtual Memory: Technique allowing processes to use more memory than physically available by keeping only active pages in RAM.
- Paging: Divides logical memory into fixed-size pages and physical memory into frames; pages can map to any frame.
- Segmentation: Divides a program into variable-length logical segments (code, data, stack) mapped independently.
- Page Fault: Exception when a needed page is not in RAM; triggers OS to load it from disk.
- Thrashing: Severe performance degradation caused by excessive page faults — the OS spends more time swapping than executing.
- Deadlock: State where two or more processes wait indefinitely for resources held by each other.
- Semaphore: Synchronization primitive — an integer variable with atomic wait (P) and signal (V) operations.
- Mutex: Binary semaphore used for mutual exclusion — only one thread can hold it at a time.
- Monolithic Kernel: All OS services in one kernel binary. Fast, but less secure/stable.
- Microkernel: Minimal kernel; most services run as user-space processes. Secure and fault-tolerant, but slower.
- Hybrid Kernel: Combines monolithic performance with microkernel modularity (e.g., macOS XNU, Windows NT).
- System Call: Interface for user programs to request kernel services; triggers mode switch from user to kernel mode.
- DMA (Direct Memory Access): Allows I/O devices to transfer data directly to/from memory without CPU intervention.
- FIFO Page Replacement: Replaces the oldest page in memory.
- LRU Page Replacement: Replaces the least recently used page.
- Optimal Page Replacement: Replaces the page not needed for the longest future time (theoretical best).
- First Fit: Allocates the first memory block that is large enough for the process.
- Best Fit: Allocates the smallest block that is large enough — minimizes wasted space.
- Internal Fragmentation: Wasted memory inside an allocated partition (allocated > needed).
- External Fragmentation: Wasted memory outside allocations — free space is scattered in small, non-contiguous chunks.
- Scheduling Algorithms: FCFS, SJF, Round Robin, Priority Scheduling — methods for deciding which process gets CPU next.
- CIA Triad: Confidentiality, Integrity, Availability — core principles of information security.

— End of Study Notes —

Bells University of Technology | CSC 307 & CSC 308 | 2025/2026